Product handover and verification guide

Batyrova, Yerkezhan | Matriculation no. 92131651 | DLBDSEDE02 Project: Data Engineering | Portfolio part 3 - Product

Repository

https://github.com/yerkebttt/municipal-air-quality-batch-ingestion

1. Clean start

```
docker compose down -v
docker compose up --build
```

The first command removes only this Compose project's database volume. The second starts MongoDB, waits for its health check, downloads the official nested UCI archive and performs an idempotent batch load.

2. Inspect health and data quality

```
docker compose run --rm ingest python -m src.monitoring
docker compose run --rm ingest python -m src.queries quality
```

A successful health report shows a terminal `succeeded` run with source, accepted, warning, rejected, inserted and existing-row counters. Source rows must reconcile. The verified fixture profile contains 420,768 rows: 383,964 accepted and 36,804 warning; no structural rejection.

3. Exercise user contracts and latency gate

```
docker compose run --rm ingest python -m src.queries trend --station Aotizhongxin
--pollutant pm25
docker compose run --rm ingest python -m src.queries compare --pollutant pm25
docker compose --profile verification up --build --exit-code-from verify
```

The benchmark checks recent readings, a daily trend, a station comparison and the quality summary against the configurable 2,000 ms target. It records evidence in `query_benchmarks` and exits nonzero on failure. Re-run ingestion once; the measurement count must not increase.

Operational boundary

`config/alert_thresholds.example.json` is non-authoritative and produces review candidates only. The prototype sends no citizen notification. Before production, replace the historical fixture with a contracted municipal feed and add verified station metadata, approved policy thresholds, access control, TLS, backups, replication and workload testing.